



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计实验报告

多线程编程实验

林晖鹏

年级：2023 级

专业：计算机科学与技术

指导教师：王刚

2025 年 5 月 21 日

摘要

本文基于 ARM 架构阿里云 ECS 实例 (Yitian 710 芯片), 本实验采用的是 8 核 cpu。代码仓库地址<https://gitee.com/absurdaaa/password-guessing.git>

本文围绕口令猜测任务, 研究并实现了基于多线程的加速方案。我们分别采用了 Pthread 与 OpenMP 两种并行方式, 针对其在不同优化等级 (O0/O1/O2) 下的执行性能进行了对比分析。实验中, 通过合理的数据结构设计 (预分配 vector 并使用索引写入), 有效缓解了并发写冲突的问题。结果表明, OpenMP 实现方式在加速效果上明显优于 Pthread, 尤其在开启编译优化 (如 O1/O2) 后, 加速比最高可达 1.5 倍。此外, 本文还从猜测总数和线程数两个维度进一步探讨了并行效率的变化趋势, 验证了任务粒度与线程调度开销之间的权衡关系。本文结合上一次 SIMD 的实验, 探究了两种并行策略下的结合效果。最后, 在原有代码框架的基础上, 本文还对口令猜测过程进行修改, 尝试一次性猜测四个口令, 将四个猜测并行化, 在不开启优化之前, 加速效果达到了惊人的 8.59 倍! 开启优化之后, 加速比可达 1.96 倍! 而破解个数仅仅下降了 18.7%。

关键字: OpenMP, Pthread, 多线程并行, 性能优化, ARM 架构, 口令猜测

目录

一、 基础要求	1
(一) pthread 多线程并行实现	1
(二) OpenMP 多线程并行实现	2
(三) 加速比与总猜测数	3
(四) 加速比与并行线程数	5
二、 进阶要求	6
(一) SIMD 和多线程同步加速	6
(二) 对哈希部分进行 SIMD 和多线程并行优化	7
(三) 多优先队列优化	8
(四) 并行优化思考	13
三、 总结	13

一、 基础要求

(一) pthread 多线程并行实现

Pthreads 是 POSIX (Portable Operating System Interface) 标准定义的用户级线程库, 用于在 C/C++ 程序中显式地创建和管理线程。它是底层、灵活、跨平台的线程编程接口 (主要用于类 Unix 系统)。

在本次实验中, 我们尝试用 pthread 进行并行优化。下面是主体代码框架, 主要是 pthread 的基本框架。

值得注意的是, 这里我们由于要把结果放入 guesses 中, 但是如果直接把单次结果直接 push_back 到 guesses 的尾部, 会导致无法并发的访问 guesses 这个变量, 并且由于要保证 guesses 中的有序性, 还得规定前后访问顺序, 这样就只能串行的访问这个 guesses, 导致效率降低。而众所周知, vector 是支持随机访问的, 这里我们使用索引访问。先把 guesses 的大小初始化为最后的大小, 然后在每次生成新的 guess 的时候, 直接放到对应位置, 这样就支持并发操作了。

```
1  int num_threads = 4; // 根据CPU核心数调整
2  int total_values = pt.max_indices[pt.content.size() - 1];
3  int init_size = guesses.size();
4  guesses.resize(guesses.size() + total_values); // 初始化 guesses 的大小
5  pthread_t threads[num_threads];
6  ThreadArgs threadArgs[num_threads]; // 构建线程函数的参数结构体
7  int chunk_size = (total_values + num_threads - 1) / num_threads;
8  for (int t = 0; t < num_threads; t++)
9  {
10     threadArgs[t].start = t * chunk_size;
11     threadArgs[t].end = min((t + 1) * chunk_size, total_values);
12     threadArgs[t].base_guess = guess;
13     threadArgs[t].a = a;
14     threadArgs[t].guesses = &guesses;
15     threadArgs[t].init_size = init_size;
16     pthread_create(&threads[t], nullptr, generate_guesses, &threadArgs[t]);
17 }
18 for (int t = 0; t < num_threads; t++) pthread_join(threads[t], nullptr);
19 total_guesses = guesses.size();
```

下面是对应具体线程函数的实现, 以及一个线程参数结构体。

```
1  // 定义线程参数结构体
2  struct ThreadArgs
3  {
4      int start;
5      int end;
6      string base_guess;
7      segment *a;
8      vector<string> *guesses;
9      int init_size;
10 };
11 // 线程函数
```

```

12 void *generate_guesses(void *args)
13 {
14     ThreadArgs *threadArgs = (ThreadArgs *)args;
15     for (int i = threadArgs->start; i < threadArgs->end; i++)
16     {
17         string temp = threadArgs->base_guess + threadArgs->a->ordered_values[
18             i];
19         (*threadArgs->guesses)[threadArgs->init_size + i] = temp;
20     }
21     pthread_exit(nullptr);
22 }

```

结果如表1所示，可以看到，在四线程的实现下，是有稍微的加速的，但是还要考虑系统的波动影响。

为啥 pthread 加速效果这么差呢，我们观察代码。在代码中，我们基本上是每 popNext 一个 PT 就创建了四个线程去并行执行任务，虽然这里是平均分的，不需要考虑忙等待的问题。但是，我们 popNext 了很多次，也就是说，我们频繁创建线程，又频繁销毁线程。这里的开销是巨大的，导致加速效果难以提高。

表 1: 结果对比

实现方式	不开优化		O1 优化		O2 优化	
	时间 (s)	加速比	时间 (s)	加速比	时间 (s)	加速比
单线程	5.06458	1.00×	0.333787	1.00×	0.300993	1.00×
四线程	4.81589	1.052×	0.305904	1.091×	0.286765	1.050×

(二) OpenMP 多线程并行实现

OpenMP 是一个用于多线程并行编程的高级 API，通过在 C/C++/Fortran 源码中加入编译指令（#pragma）来实现多线程功能。常用于科学计算、数据并行等 CPU 密集型任务。

OpenMP 的使用比 pthread 就简单很多了，下面是核心代码部分，我们这里同样用索引访问来解决并发访问的问题。

```

1  int init_size = guesses.size();
2  //这里获取了最多能设置的线程数
3  int max_threads = omp_get_max_threads();
4  guesses.resize(guesses.size() + pt.max_indices[pt.content.size() - 1]);
5  #pragma omp parallel num_threads(max_threads)
6  {
7  #pragma omp for
8      for (int i = 0; i < pt.max_indices[pt.content.size() - 1]; i++)
9      {
10         string temp = guess + a->ordered_values[i];
11         //直接索引写入
12         guesses[init_size + i] = temp;
13     }
14 }

```

15

```
total_guesses = guesses.size();
```

实验结果如表2所示。可以看到，OpenMP 的加速效果在开了优化之后是比较明显的。

为啥 openmp 比 pthread 效果好呢，同样是频繁调用了线程。其实 openmp 虽然也是每次到达这部分代码就调用并行线程，但是它只会第一次调用的时候创建，然后不使用并行的时候把线程挂起，等到下次使用并行的时候就不需要重新创建线程了，虽然会损失一定空间，但是微乎其微。所以 openmp 能达到好一点的加速效果。

表 2: 结果对比

实现方式	不开优化		O1 优化		O2 优化	
	时间 (s)	加速比	时间 (s)	加速比	时间 (s)	加速比
单线程	5.06458	1.00×	0.333787	1.00×	0.300993	1.00×
四线程	4.73815	1.069×	0.260092	1.283×	0.238475	1.262×

(三) 加速比与总猜测数

下面，我们来对不同猜测数下的时间进行对比，这里我们把猜测数上限改成一亿，然后在不同优化设置下进行对比。

这里我们一方面增加总猜测数的上限，另一方面只是在多个猜测数节点处记录时间，由于记录时间的开销，这里的总时间会变长，但是不同方式下的开销增加是相同的，而根据糖水不等式，这样只会让加速比减小而不是增大，况且这个开销相当小，可以忽略不计。

不开优化的结果如图1所示，可以发现，在一千万猜测数之内的时候，并行方法的加速比都是在上升趋势的，而猜测数到达一千五百万之后，加速比趋于稳定，分别稳定在 **1.084** (pthread) 和 **1.121** (OpenMP)。

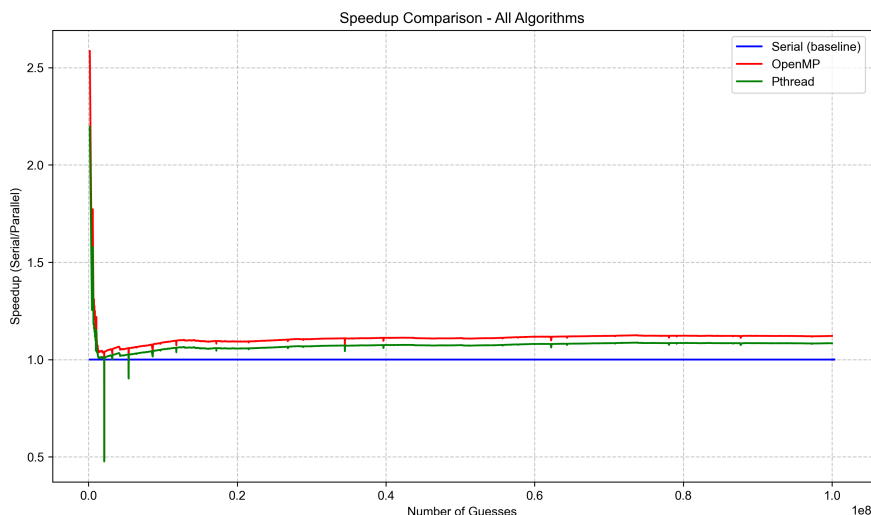


图 1: O0

开启 O1 优化的结果如图2所示，可以发现开启优化之后，加速更加明显了，并且稳定的更慢，直到猜测数到达五千万的时候才趋于稳定。此时加速比稳定在 **1.206** (pthread) 和 **1.500** (OpenMP)。

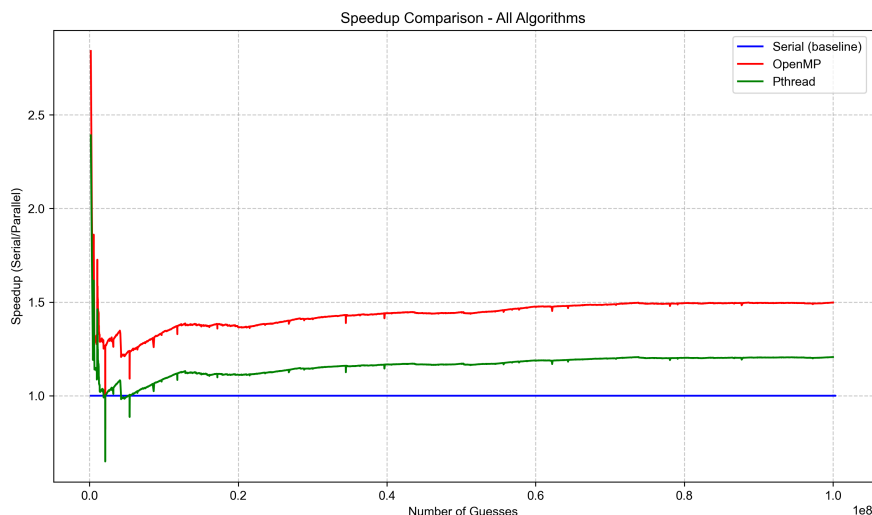


图 2: O1

开启 O2 优化的结果如图3所示，开启 O2 优化之后，发现反而加速比还下降了一点，说明开启 O2 优化之后，对串行版本的方法优化力度更大，导致基线变高了。而在 O2 版本下，趋于稳定的节点更加大了，甚至在图中还仍在上升趋势。此时加速比在 1.164 (pthread) 和 1.460 (OpenMP)。

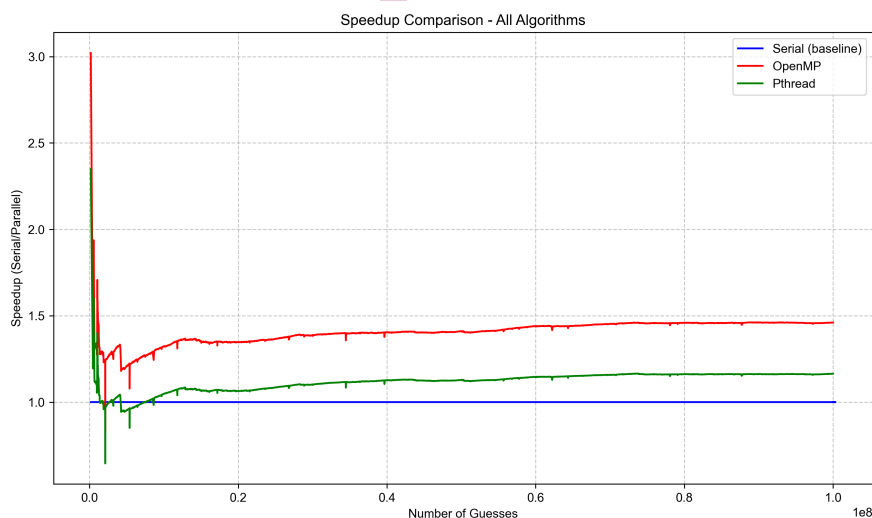


图 3: O2

• 小结与分析

在上面的对比中，我们发现了一个现象：随着猜测数变大，加速比先变大后稳定。

我们知道，并行任务的创建是有开销的。在使用多线程时，每次并行任务启动时有这些额外开销：线程创建/调度的时间、数据划分与任务分发、线程同步与加锁、CPU 缓存一致性维护、false sharing 等低层性能损耗。

而在猜测量比较小的时候，这些开销占大头，分摊到每个猜测计算中的时间比较大，导致加速比较小；当猜测量比较大的时候，这些开销有一部分是固定，这些固定开销被均摊之后特别小，在加速比计算中就可以忽略不计了，所以后面的加速比就趋于稳定了。

所以说，pthread 这个方法是确实实现加速的，但是在猜测量一千万的时候体现并不是很明显。

(四) 加速比与并行线程数

我们来探究一下并行线程数对加速比的影响。这里我们在开启 O2 优化的情况下，分别对 pthread 和 OpenMP 两种方法在线程数为 1~8 的情况下进行实验，得到结果如表3所示：（这里我服务器的 cpu 核心为 8，而主线程只有分配任务的使用，所以线程数这里可以开到 8，不存在超出物理核心数的情况。）

表 3: 不同并行线程数结果

线程数	Pthread-O2		OpenMP-O2	
	时间 (s)	加速比	时间 (s)	加速比
1 线程	0.392677	0.77×	0.324796	0.93×
2 线程	0.309910	0.97×	0.260703	1.15×
3 线程	0.283852	1.06×	0.233602	1.29×
4 线程	0.281583	1.07×	0.222322	1.35×
5 线程	0.303205	0.99×	0.216787	1.39×
6 线程	0.311211	0.97×	0.210932	1.43×
7 线程	0.325405	0.92×	0.207917	1.45×
8 线程	0.344234	0.87×	0.218134	1.38×

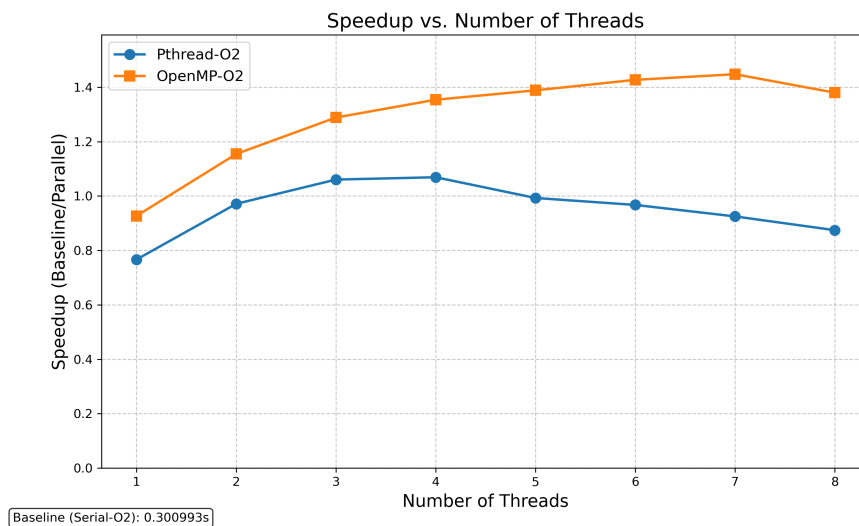


图 4: 不同并行线程数加速比对比

可以看到，在线程数为 1 的时候，并行优化都是负优化，因为存在设置并行的开销。

而当并行线程数增加的时候，加速比开始上升，并行带来的加速盖过了并行的额外花销，实现了并行加速。

但是当并行线程数增加到一定数量的时候，比如这里 pthread 是 5 之后，openmp 是 8 之后，加速比开始下降，发生了并行加速“反转”。是因为：

- 当线程数过大的时候，分配任务、等待同步的时间反而占了主导，导致加速比下降。
- 多线程会竞争 L1L2L3 缓存的使用，结合我这里用的是随机访问，会导致访问的时候反复大面积跳转，增加了开销。

可以观察到，pthread 在较小线程数时加速比上升较快，但很早就到达瓶颈，其原因主要是：

- 线程管理开销大：pthread 每次都需显式创建、销毁线程，系统调用成本高，缺乏线程池机制；
- 任务划分与调度手动完成：划分不均或任务粒度过细会造成负载不均，影响性能；
- 缺乏自动优化能力：pthread 不具备 OpenMP 的自动调度、线程复用、核心绑定等机制；

相比之下，OpenMP 使用线程池并具备动态调度能力，因此能在更高线程数下保持较好的加速效果。

二、进阶要求

(一) SIMD 和多线程同步加速

下面我们把上次实验的 SIMD 四路并行和这次实验的 OpenMP 多线程并行优化结合（这里我们选用效果最好的 7 线程数）。下面是运行结果：

表 4: SIMD 和多线程同步加速结果

优化程度	无优化版本			
	Hash 时间 (s)	加速比	Guess 时间 (s)	加速比
O0	11.7818	1.00×	5.06458	1.00×
O1	6.44946	1.00×	0.333787	1.00×
O2	6.29782	1.00×	0.300993	1.00×
优化程度	SIMD+OpenMP			
	Hash 时间 (s)	加速比	Guess 时间 (s)	加速比
O0	11.3187	1.04×	4.77009	1.06×
O1	4.36641	1.48×	0.230478	1.45×
O2	4.31747	1.46×	0.215457	1.40×

虽然实验指导手册上面说可能不能同时实现优化，但是我们惊喜的发现，在同时优化的情况下，仍然进行了加速（归功于我们选取了最优的多线程数）。

可以看到，不同优化力度下的加速结果是不同的。

在无优化的情况下，加速效果是微乎其微的，但是正如我们前面分析猜测数和加速比的关系所分析的，其实这里如果增加计算量，加速比应该是能够更加大一点，所以这里稍微加速肯定不是系统波动的结果，而是确实确实加速了。

可以看到，O1 优化下的加速比是最大的，无论是 SIMD 还是 OpenMP，而 O2 优化下反而加速比小了一点，因为不同优化下对两个版本代码的优化效果不同。

这里需要解释一下，为什么 Hash 时间对比上一实验差别这么多。因为这里加了一个破解率的检测，但是根据糖水不等式，这里 hash 时间的加速比只会比原来的小而不会变大：

$$\frac{\text{无优化 hash 时间}}{\text{并行优化后 hash 时间}} \geq \frac{\text{无优化 hash 时间} + \text{破解率检测时间}}{\text{并行优化后 hash 时间} + \text{破解率检测时间}}$$

(二) 对哈希部分进行 SIMD 和多线程并行优化

两次实验下来，我们可以发现，SIMD 和多线程优化是不同的优化方向，SIMD 是一条代码情况下实现多个计算量同时计算，而多线程是同一时间下实现多条代码的同时计算。而在之前实验中，我们只对 Hash 部分实现了 SIMD，现在，我们尝试在 SIMD 的基础上用 openmp 实现多线程优化（这里同样考虑使用线程数为 7）。

值得注意的是，由于这个 Md5Hash 部分是为了包饺子，所以这里原本代码并没有对结果进行存储，如果加上存储，这个存储还需要像之前 resize 那个 guesses 一样，对结果进行随机方法，来解决并行访问的问题。

由于不需要存储结果，所以这里只需要考虑对 cracked 这个同时做加法的问题，我们这里使用原子操作，来解决并发更改 cracked 的问题。

```

1      alignas(16) bit32 state[4][4];
2      string pw[4];
3
4      // 使用 OpenMP 并行化外层循环
5      #pragma omp parallel for num_threads(7) shared(q, test_set) private(pw,
6          state)
7          for (int i = 0; i < q.guesses.size(); i += 4)
8          {
9              for (int j = i; j < q.guesses.size() && (j - i) < 4; j++)
10             {
11                 pw[j - i] = q.guesses[j];
12                 if (test_set.find(pw[j - i]) != test_set.end())
13                 {
14                     #pragma omp atomic
15                     cracked += 1; // 使用原子操作保证线程安全
16                 }
17             }
18             SIMD_MD5Hash(pw, state);
19         }

```

表 5: 结果对比（基于 Hash time）

实现方式	不开优化		O1 优化		O2 优化	
	时间 (s)	加速比	时间 (s)	加速比	时间 (s)	加速比
无优化	11.7818	1.00×	6.44946	1.00×	6.29782	1.00×
SIMD4	11.3559	1.04×	4.40773	1.46×	4.26419	1.48×
SIMD+OpenMP	1.53559	7.67×	0.578018	11.15×	0.593664	10.61×

结果如表所示，可以看到，同时在 SIMD 的数据级并行优化以及 OpenMP 的线程级并行优化下，加速比最好达到了 11.15 倍!!!

加速了不奇怪，但是值得思考的是，为什么我只是使用了七个线程数，但是比 SIMD 优化快了七倍以上。

下面我们用 perf 来进行分析对比：

表 6: perf 指标对比

指标	SIMD	SIMD+OpenMP	变化量	变化率
CPU 利用率	1.013	1.140	+0.127	+12.54%
缓存访问次数	211,795,780,779	211,469,692,093	-326,088,686	-0.15%
缓存未命中次数	798,629,688	749,899,449	-48,730,239	-6.10%
缓存未命中率	0.38%	0.35%	-0.03%	-7.89%
分支预测未命中率	1.48%	1.47%	-0.01%	-0.68%
每周期指令数 (IPC)	1.51	1.52	+0.01	+0.66%

可以看到：

- SIMD 的 CPU 利用率为 1.013，而 SIMD+OpenMP 为 1.140，增加了 12.54%。这表明 OpenMP 有效地利用了多核 CPU 资源，提升了并行计算效率。
- SIMD 的缓存未命中率为 0.38%，而 SIMD+OpenMP 为 0.35%，减少了 7.89%，虽然看着减少了一点点，但是这可是所有部分加起来的缓存未命中率，从这个结果可见一斑，Hash 部分的缓存未命中率应该是大大减小了。
- SIMD 的 IPC 为 1.51，而 SIMD+OpenMP 为 1.52，增加了 0.66%。IPC 的小幅提升表明 OpenMP 的并行化提高了指令执行效率。

可以看到，openmp 不仅仅只是对并行部分代码进行了并行化，同时优化了缓存性能，使得加速效果大于了线程数。

(三) 多优先队列优化

下面我们尝试对生成猜测以及 PT 优先队列过程进行进一步的 OpenMP 优化。

首先我们观察到，在 init 函数中，遍历了一次 m.ordered_pts，然后把每个 pt 拿出来，初始化之后，计算概率值，然后放进一个 vector 里面，形成一个优先队列，但是这里其实是可以并行化的。我们把这个遍历过程分成几个线程，每个线程存储各自的优先队列，然后最后在合并，核心代码如下：

这里写了一个对多个优先队列归并排序的函数，在对上面所说的遍历过程并行化之后，在归并排序他们的结果。

显然这里的加速过程不会很明显，因为 init 中遍历的 PT 数不会特别多。

```

1 std::vector<PT> merge_vectors(const std::vector<PT> &vec1, const std::vector<
  PT> &vec2)
2 {
3     if (vec1.empty())
4         return vec2;
5     if (vec2.empty())
6         return vec1;
7

```

```

8   std::vector<PT> merged(vec1.size() + vec2.size());
9   std::merge(vec1.begin(), vec1.end(), vec2.begin(), vec2.end(), merged.begin
    (),
10          [](const PT &a, const PT &b)
11          { return a.prob > b.prob; }); // 按概率降序排序
12   return merged;
13 }
14
15 // 并行归并排序
16 std::vector<PT> parallel_merge_sort(std::vector<std::vector<PT>> &
    local_queues)
17 {
18     // 如果为空或只有一个队列，直接返回
19     if (local_queues.empty())
20     {
21         return std::vector<PT>();
22     }
23     if (local_queues.size() == 1)
24     {
25         return local_queues[0];
26     }
27
28     // 使用分治法合并队列
29     while (local_queues.size() > 1)
30     {
31         // 创建临时向量存储合并结果
32         std::vector<std::vector<PT>> temp_queues((local_queues.size() + 1) / 2);
33         int new_size = temp_queues.size();
34
35         // 并行合并相邻队列
36         #pragma omp parallel for schedule(static)
37         for (int i = 0; i < new_size; i++)
38         {
39             if (2 * i + 1 < local_queues.size())
40             {
41                 // 安全地合并两个队列到临时向量
42                 temp_queues[i] = merge_vectors(local_queues[2 * i], local_queues[2 *
                    i + 1]);
43             }
44             else
45             {
46                 // 处理奇数情况
47                 temp_queues[i] = local_queues[2 * i];
48             }
49         }
50
51         // 将临时向量交换到原向量
52         local_queues = std::move(temp_queues);

```

```

53     }
54
55     // 返回最终合并的队列
56     return local_queues.empty() ? std::vector<PT>() : local_queues[0];
57 }
58
59 void PriorityQueue::init()
60 {
61     // 获取线程数
62     int num_threads = omp_get_max_threads();
63
64     // 每个线程维护一个本地优先队列
65     std::vector<std::vector<PT>> local_queues(num_threads);
66
67     // 并行化处理 m.ordered_pts
68     #pragma omp parallel num_threads(num_threads)
69     {
70         int thread_id = omp_get_thread_num();
71         std::vector<PT> &local_queue = local_queues[thread_id];
72
73         // 分块处理 m.ordered_pts
74         #pragma omp for schedule(static)
75         for (size_t i = 0; i < m.ordered_pts.size(); i++)
76         {
77             PT pt = m.ordered_pts[i];
78
79             // 处理每个 PT 的内容
80             for (segment seg : pt.content)
81             {
82                 if (seg.type == 1)
83                 {
84                     pt.max_indices.emplace_back(m.letters[m.FindLetter(seg)].
                        ordered_values.size());
85                 }
86                 if (seg.type == 2)
87                 {
88                     pt.max_indices.emplace_back(m.digits[m.FindDigit(seg)].
                        ordered_values.size());
89                 }
90                 if (seg.type == 3)
91                 {
92                     pt.max_indices.emplace_back(m.symbols[m.FindSymbol(seg)].
                        ordered_values.size());
93                 }
94             }
95
96             // 计算 PT 的概率
97             pt.preterm_prob = float(m.preterm_freq[m.FindPT(pt)]) / m.total_preterm

```

```

    ;
98     CalProb(pt);
99
100    // 将 PT 添加到本地优先队列
101    local_queue.emplace_back(pt);
102    }
103    }
104    // 使用并行归并排序合并所有线程的本地优先队列
105    priority = parallel_merge_sort(local_queues);
106    }

```

然后我们在来修改 PopNext 函数，每次只对一个 PT 生成猜测并生成新的猜测 PT 太慢了，我们希望这里能一次性处理 PT 优先队列中的前几个 PT，虽然会导致猜测不是最优的，因为在原本代码中，下一个要生成猜测的 PT 不一定是第二个，可能是新插入的 PT。但是这里我们选择牺牲一些颗粒度，后面结果显示，这个牺牲是完完全全值得的。

然后每次插入新的 PT 时，都是从头开始遍历优先队列，找到合适位置在插进去，复杂度极高！

我们考虑一次性处理前四个 PT，每次处理都并行化的生成猜测，并行化的生成各自的新 PT 们，并并行化的对各自的新 PT 们用 sort 进行排序，得到个各自的优先队列，然后在用上面设计的归并排序函数，把这几个优先队列，以及原本的优先队列进行合并。代码如下：

在设计代码时，要注意几点：

- 有些地方 openmp 分配任务容易不均匀，会导致忙等待，所以在计算各自的优先队列的时候，我们使用了动态分配任务，防止一些队列比较大，导致计算概率比较久。
- 这里如果嵌套使用 openmp，容易导致资源竞争，而且默认是不允许嵌套使用 openmp 的，这里我们简单点也不嵌套了，所以 Generate 函数里面是原版串行的函数。只是把 guess 改成了本地存储的，而不是全局的（避免并发访问）。

```

1 void PriorityQueue::PopNext4()
2 {
3     int NUM_THREADS = 4;
4
5     vector<vector<PT>> new_pts(NUM_THREADS);
6     vector<string> local_guesses [NUM_THREADS];
7     int local_num [NUM_THREADS] = {0};
8
9     #pragma omp parallel for schedule(static)
10    for (int i = 0; i < NUM_THREADS; i++)
11    {
12        PT pt;
13        // 线程安全地获取队首元素
14        #pragma omp critical
15        {
16            if (!priority.empty())
17            {
18                pt = priority.front();
19                priority.erase(priority.begin());

```

```

20     }
21 }
22
23 // 如果成功获取到队首元素, 则进行处理
24 if (!pt.content.empty())
25 {
26     Generate(pt, local_guesses[i], local_num[i]);
27     new_pts[i] = pt.NewPTs();
28 }
29 }
30
31
32 // 合并猜测结果
33 for(int i = 0; i < NUM_THREADS; i++){
34     total_guesses += local_num[i];
35 }
36 guesses.resize(total_guesses);
37 for(int i = 0; i < NUM_THREADS; i++){
38     guesses.insert(guesses.end(), local_guesses[i].begin(), local_guesses[i].
39         end());
40 }
41
42 // 这里我先把各自排成优先队列之后在合并
43 // 动态分配任务
44 #pragma omp parallel for schedule(dynamic)
45 for (int i = 0; i < NUM_THREADS; i++)
46 {
47     // 遍历当前线程的 new_pts[i], 计算概率
48     for (PT &pt : new_pts[i]) CalProb(pt);
49     // 对当前线程的 new_pts[i] 按概率降序排序
50     std::sort(new_pts[i].begin(), new_pts[i].end(), [](const PT &a, const PT
51         &b)
52     {
53         return a.prob > b.prob; // 按概率降序排序
54     });
55 }
56
57 // 先合并新生成的
58 vector<PT> all_new_pts = parallel_merge_sort(new_pts);
59 // 然后在合并到优先队列中
60 priority = merge_vectors(priority, all_new_pts);
61 }

```

表 7: 超级并行优化结果对比

实现方式	不开优化		O1 优化		O2 优化	
	时间 (s)	加速比	时间 (s)	加速比	时间 (s)	加速比
无优化	5.06458	1.00×	0.333787	1.00×	0.300993	1.00×
openmp 优化	4.73815	1.07×	0.260092	1.28×	0.238475	1.26×
超级 OpenMP 优化	0.5894	8.59×	0.170608	1.96×	0.168509	1.79×

结果如表7所示，超级夸张，在记录数据的时候一度怀疑是不是写错了，但是反复确认了一下，同一优化下 Train time 是差不多的。

在超级 OpenMP 优化中，破解个数仅仅下降到了 **291267**，减少了 **18.7%**，但是加速比是相当可以的。甚至在不开优化的时候，加速比到达了 **8.59x**，也应证了我前面的观察，这里估计是因为反复对 PT 插入排序导致时间花销巨大。

但是在开启优化之后，这个加速比的差距被大大降低，可以合理猜测，这里反复的插入排序被优化掉了。

(四) 并行优化思考

1. 并行化适合处理什么样的问题？

并行实际上是对多余资源的充分利用，但是同时会产生线程调度开销、同步与通信的成本、负载不均、资源争夺的问题。所以并不是所有任务都适合并行的，一旦这些问题产生的额外开销盖过了并行带来的优化，那么这个并行又耗时间又耗费计算资源。

所以，并行化适合处理一些计算密集型任务、可分解问题型、大批量数据的任务。这些任务往往可以分解成诸多个子任务，并且任务之间的相互依赖关系比较弱。对这些问题并行化，能够充分利用计算资源的同时加快程序运行。

2. 什么时候并行化更合适？什么时候串行更合适？

并行和串行往往不是非黑即白的，而是相互配合。

当任务存在较高的相互依赖性的时候，此时同步和通信带来的时间花销容易超过并行带来的优化，这时候就应该选择使用串行化。

而当任务中存在一部分代码，他们的前后依赖性比较低的时候，此时并行化是一个很好的选择。

当然，不是一遇到无依赖可分解的任务就直接并行化，分解的子任务最好比较大，否则，频繁的调用并行线程，线程调度的开销同样会让并行化得不偿失，反而比串行更差。

还需要考虑的是计算资源的大小，如果分解后的并行子任务对计算资源的要求还是很大，那么并行分配给这个子任务的计算资源可能不够，可能导致资源争夺的情况发生。（比如，每个任务只有一个 cpu，那么每个任务的缓冲区就会很小，导致缓存命中率低，可能导致执行效率下降）。

三、 总结

在本次并行口令猜测实验中，我们分别采用了 Pthread 和 OpenMP 两种主流多线程方法，系统地比较了它们在不同编译优化等级、不同猜测上限和不同线程数量下的性能表现。

从实验数据可知, OpenMP 在本实验场景中明显优于 Pthread: 前者在保证线程复用的前提下, 通过静态线程池机制极大降低了线程创建/销毁开销, 结合编译优化选项 (如-O1、-O2), 加速比最高达到了大约 $1.5\times$; 而 Pthread 虽然在理论上可实现并行加速, 但由于其每次任务分配都涉及线程生命周期管理, 导致在小规模任务下性能提升不明显, 加速比仅约 $1.1\times$ 上下浮动。

此外, 实验还揭示了并行任务的开销在总猜测数较小时占比较大, 造成初期加速比不显著; 而随着任务规模提升, 这部分固定开销被均摊, 最终加速比趋于稳定。线程数实验进一步表明, 在合理利用物理核心数的前提下, OpenMP 的并行效率具备良好的线性扩展趋势。

本实验还结合了 SIMD 和多线程两种并行化策略, 发现 SIMD 是数据级并行化, 多线程是线程级并行化, 两种并行化结合下, 能进一步加速。

本实验最后尝试, 牺牲口令的部分排序, 只达到整体排序, 一次性猜测四个 PT, 将猜测过程并行化, 最终达到最好效果。

综上所述, OpenMP 更适合本类 CPU 密集型、任务结构规则的场景, 未来在其他密码学计算、高性能计算等场景中具有广泛推广价值。而 Pthread 更适合对线程控制粒度要求高、任务高度异构的系统。

MINU